
PROBABILISTIC FORECASTING VIA AUTOREGRESSIVE FLOW MATCHING

Anonymous authors
Anonymous submission

ABSTRACT

We introduce FLOWTIME, a generative framework for probabilistic forecasting in multivariate timeseries. Given historical observations and optional future covariates, the model casts forecasting as conditional sampling from the distribution of future trajectories. Rather than transporting an entire prediction window as a single high-dimensional object, FLOWTIME factorizes the future joint distribution into sequential conditional densities and models each density with a shared flow that maps a simple base distribution to the next-step observation distribution under the observed covariates. The resulting formulation brings flow matching (FM) to autoregressive forecasting, yielding scalable, simulation-free training of these conditional transformations. This combination preserves the central advantages of autoregressive forecasting—effective extrapolation, compact parameterization, and calibrated uncertainty propagation—while exploiting the expressivity of transport-based generative models for complex conditional distributions. Experiments on stochastic dynamical systems and real-world forecasting benchmarks show that FLOWTIME delivers strong probabilistic forecasts and, in the evaluated settings, improves or matches competitive flow-matching and diffusion-based baselines.

1 INTRODUCTION

A central problem in modern machine learning is probabilistic timeseries forecasting: estimating how system trajectories may evolve beyond the observed record while retaining a quantified description of predictive uncertainty. This formulation is fundamental across scientific, industrial, and societal domains in which decisions depend not only on point predictions but also on the distribution of plausible future outcomes (Lim & Zohren, 2021; Dama & Sinoquet, 2021; Ye et al., 2024).

Deep generative models provide a natural route to this problem by representing forecasting as sampling from a future probability density conditioned on the past (Karl et al., 2016; Rasul et al., 2020; Desai et al., 2021). Within this line of work, diffusion and score-based generative models (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2020) have substantially advanced performance on forecasting benchmarks (Rasul et al., 2021; Tashiro et al., 2021; Kollovich et al., 2023; Meijer & Chen, 2024). Their effectiveness, however, is often accompanied by substantial computational cost, because both training and inference typically require many stochastic denoising operations.

Flow Matching (FM) (Liu et al., 2022; Albergo et al., 2023; Lipman et al., 2022) offers a complementary generative paradigm that generalizes diffusion-style constructions while allowing more flexible choices of probability paths. Instead of learning an iterative stochastic denoising process over a discretized trajectory, FM learns a deterministic vector field whose continuous flow transports an arbitrary base distribution to the target distribution. With appropriately chosen probability paths, FM avoids handcrafted noise schedules and long ancestral sampling chains, thereby supporting more efficient training and generation.

Recent forecasting methods have begun to exploit FM for timeseries modeling (Tamir et al., 2024; Hu et al., 2024; Kollovich et al., 2024), obtaining strong empirical performance and improved computational efficiency relative to diffusion-based alternatives. Existing FM forecasting approaches nevertheless typically learn the conditional distribution of an entire fixed-horizon future window given a fixed-horizon context window. This non-autoregressive formulation can be fast, but it also concentrates the modeling burden into a high-dimensional trajectory-level transport problem, can

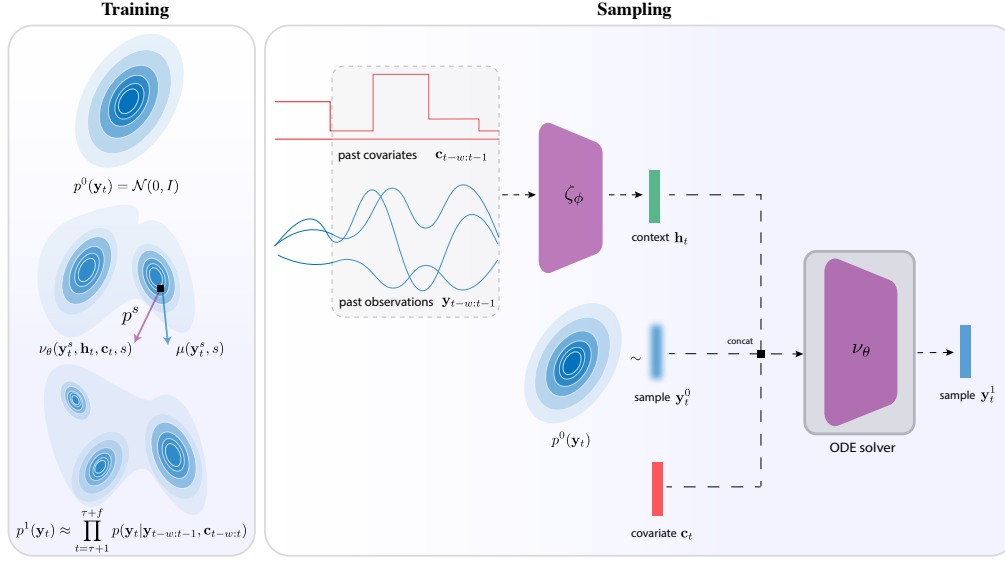


Figure 1: Overview of the proposed training and sampling procedure. During training, a probability path $(p^s)_{0 \leq s \leq 1}$ connects the base distribution p^0 to the target distribution p^1 through a velocity field μ . The neural velocity field ν_θ is trained to regress this target velocity from a path sample $\mathbf{y}_t^s$ at flow step s , a context representation $\mathbf{h}_t$ of past observations and covariates, and the current covariate $\mathbf{c}_t$. At inference time, sampling begins from p^0 and integrates the learned velocity field with an ODE solver until $s = 1$, producing a draw from the next-step conditional distribution.

extrapolate poorly outside the training horizon, and may yield miscalibrated uncertainty estimates. The optimization difficulty is explicit in (Kollovieh et al., 2024), which introduces Gaussian-process-informed priors to simplify and accelerate learning, but at the cost of restricting the method to univariate forecasting.

We propose FLOWTIME, an autoregressive FM model for probabilistic forecasting that scales to high-dimensional multivariate datasets without informed priors. The key technical move is to replace full-window transport with a sequence of next-step conditional transports: each future observation is sampled from a conditional density determined by a finite history window and the relevant covariates, and all such densities are represented through a shared flow from a simple base distribution. This factorization aligns FM with the temporal structure of forecasting, reduces the dimensionality of each learned transport, supports variable forecast horizons, and makes uncertainty propagation explicit through the chain of conditional distributions.

The paper contributes an autoregressive flow-matching formulation for probabilistic forecasting, a training and sampling procedure that preserves parallel teacher-forced training while enabling sequential forecast generation, and an empirical evaluation on simulated stochastic dynamical systems and real-world datasets. Across these settings, the results show that FLOWTIME is a practical and competitive approach for probabilistic timeseries forecasting, with particularly clear benefits for extrapolation and next-step accuracy in the reported dynamical-system experiments.

2 BACKGROUND: FLOW MATCHING

Flow matching (FM) (Lipman et al., 2022) is a generative-modeling framework for training continuous normalizing flows (CNFs) (Chen et al., 2018; Grathwohl et al., 2018) at large scale. CNFs represent transformations between probability distributions as solutions of ordinary differential equations (ODEs), yielding invertible maps with tractable likelihoods.

As in CNFs, FM seeks a time-dependent diffeomorphic map on the data space Ω , denoted by the flow $\psi: [0, 1] \times \Omega \rightarrow \Omega$. The flow transports a source sample $\mathcal{X}^0 \sim p$ to a target sample $\mathcal{X}^1 := \psi(\mathcal{X}^0, 1)$

such that $\mathcal{X}^1 \sim q$ for a target distribution q . This map is defined through the initial value problem

$$d\psi(\mathcal{X}, s) = \mu(\psi(\mathcal{X}, s), s)ds, \quad \psi(\mathcal{X}, 0) = \mathcal{X}, \quad s \in [0, 1] \quad (1)$$

where $\mu: [0, 1] \times \Omega \rightarrow \Omega$ is the velocity field that generates a probability path $(p^s)^{0 \leq s \leq 1}$ over Ω with initial marginal $p^0 = p$. The learning problem is therefore to identify a vector field whose terminal marginal satisfies $p^1 = q$.

Whereas standard CNFs learn the vector field by likelihood maximization and therefore require solving and differentiating through Eq. (1) during training, FM estimates a parameterized vector field ν_θ by regression against vector fields associated with fixed conditional probability paths. This construction yields the tractable conditional flow matching (CFM) objective

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{s \sim \mathcal{U}[0,1], \mathcal{X}^1 \sim q(\mathcal{X}^1), \mathcal{X} \sim p^s(\mathcal{X}|\mathcal{X}^1)} \|\nu_\theta(\mathcal{X}, s) - \mu(\mathcal{X}, s|\mathcal{X}^1)\|^2. \quad (2)$$

After optimization, samples from q are obtained by solving Eq. (1) with the learned field ν_θ . Because the training objective avoids backpropagation through the ODE solver, FM supports stable and scalable learning in high-dimensional generative settings, including image and video modeling (Esser et al., 2024; Polyak et al., 2024). Different probability-path constructions and conditioning schemes further place several transport-based generative models within the same framework (Liu et al., 2022; Albergo et al., 2023; Tong et al., 2023).

3 AUTOREGRESSIVE FLOW MATCHING

Problem Setting Let $\mathbf{y}_\tau \in \mathbb{R}^n$ denote an n -dimensional observation at time τ . Given the most recent l observations, including the current observation, $\mathbf{Y}_l = \{\mathbf{y}_{\tau-l}, \dots, \mathbf{y}_\tau\}$, the forecasting task is to sample the next f values $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$ from their predictive distribution. The model may additionally condition on covariates $\mathbf{C} = \{\mathbf{c}_{\tau-l}, \dots, \mathbf{c}_{\tau+f}\}$, where $\mathbf{c}_\tau \in \mathbb{R}^c$. Formally, given $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$, we aim to learn a sampler for $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$.

FLOWTIME factorizes this conditional distribution autoregressively across future time steps under a Markov assumption of order w :

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \quad (3)$$

where $w \leq l$ is the history size. The modeling objective is consequently reduced to learning the next-step conditional distributions in Eq. (3). Rather than fitting these conditionals by maximum likelihood, as in many autoregressive forecasting models, we estimate them with the flow matching framework (Lipman et al., 2022).

This factorization is the principal structural contribution of the method. It replaces a single high-dimensional trajectory distribution with lower-dimensional conditional transports $p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$, while still defining a joint predictive distribution through their product. It also permits teacher-forced training of individual time steps in parallel, exposes a direct capacity-complexity trade-off through the window size w , and makes temporal uncertainty propagation explicit because each sampled prediction becomes part of the conditioning context for subsequent horizons.

3.1 CONSTRUCTING A TARGET PROBABILITY PATH

For each future time step $t \in [\tau+1, \tau+f]$, we specify a probability path $(p^s(\mathbf{y}_t))_{0 \leq s \leq 1}$ with source marginal $p^0(\mathbf{y}_t) = \mathcal{N}(0, I)$ and terminal marginal $p^1(\mathbf{y}_t) \approx \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \approx p_{\mathcal{D}}$, where $p_{\mathcal{D}}$ denotes the empirical distribution. The path is generated by a flow field $\psi: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ along the flow variable s according to the ODE

$$d\psi(\mathbf{y}_t, s) = \mu(\psi(\mathbf{y}_t, s), s)ds, \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t \quad s \in [0, 1] \quad (4)$$

where $\mu: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ is the velocity field selected so that the induced probability path satisfies the continuity equation.

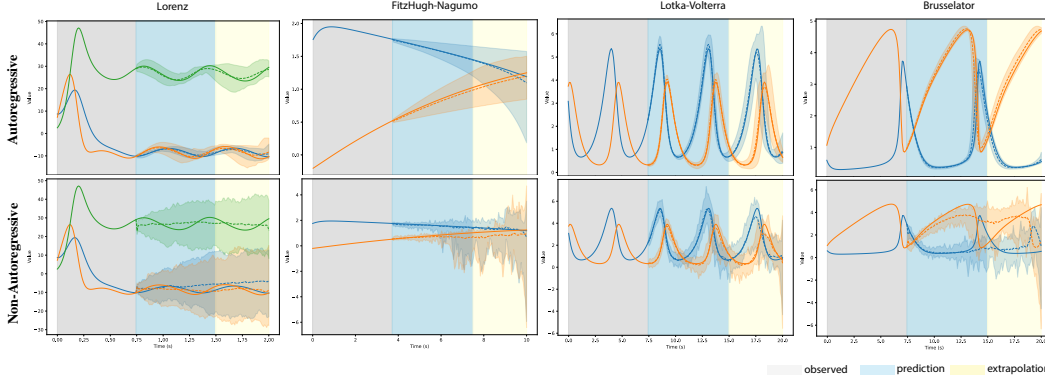


Figure 2: Forecast samples comparing autoregressive flow matching with standard flow matching on four stochastic dynamical systems. Solid curves denote ground truth, and dashed curves denote the predictive mean with the 95% confidence interval. Each panel reports behavior in both the prediction and extrapolation regimes.

To obtain a tractable learning objective, we express the marginal probability path as a mixture of conditional paths:

$$p^s(\mathbf{y}_t) = \int p^s(\mathbf{y}_t|\mathbf{z})p(\mathbf{z})d\mathbf{z} \quad (5)$$

where $\mathbf{z} \sim \pi_{0,1}(\mathbf{z})$ is a conditioning variable sampled from a data coupling $\pi_{0,1}$. Theorem 2 of (Lipman et al., 2022) establishes that regressing the conditional velocity fields $\mu(\mathbf{y}_t, s|\mathbf{z})$ associated with $p^s(\mathbf{y}_t|\mathbf{z})$ recovers the corresponding marginal velocity field $\mu(\mathbf{y}_t, s)$.

Following (Tong et al., 2023), we set $\mathbf{z} = \{\mathbf{y}_t^0, \mathbf{y}_t^1\}$ with $p(\mathbf{z} = (\mathbf{y}_t^0, \mathbf{y}_t^1)) = p^0 \times p^1$, where $\mathbf{y}_t^0$ and $\mathbf{y}_t^1$ are sampled from p^0 and p^1 , respectively. The conditional path is then

$$p^s(\mathbf{y}_t|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I) \quad (6)$$

with endpoint marginals satisfying $p^0(\mathbf{y}_t|\mathbf{z}) = p^0(\mathbf{y}_t)$ and $p^1(\mathbf{y}_t|\mathbf{z}) = p^1(\mathbf{y}_t)$ as $\sigma^2 \rightarrow 0$. The associated conditional velocity field is $\mu(\mathbf{y}_t, s|\mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$. This construction yields straight Euclidean probability paths and closed-form target velocities, which are central to the computational simplicity of the method. Once the velocity field is learned, samples from $p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ are obtained by solving the ODE in Eq. (4) until $s = 1$.

3.2 TRAINING

As in standard FM, we learn the velocity field by fitting a neural approximation ν_θ with parameters θ using the conditional flow matching objective in Eq. 2. To condition this field on the autoregressive history, we introduce a context vector $\mathbf{h}_t \in \mathbb{R}^h$ for time t :

$$\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1}) \quad (7)$$

where $\zeta: \mathbb{R}^{w \times n} \times \mathbb{R}^{w \times c} \rightarrow \mathbb{R}^h$ is a neural encoder with parameters ϕ . The resulting learning objective is

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{y}_t^s \sim p^s(\mathbf{y}_t|\mathbf{z})} \|\mu(\mathbf{y}_t^s, s|\mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2 \quad (8)$$

Training samples an observation at time t , its context window, and its covariates from $\mathcal{D}$, and jointly optimizes ϕ and θ by minimizing $\mathcal{L}(\theta, \phi)$ with stochastic gradient descent. This procedure is teacher-forced (Williams & Zipser, 1989): ground-truth past observations, rather than previously generated predictions, are supplied during training. Consequently, all training time steps can be processed in parallel and training-time error accumulation is avoided. Algorithm 1 summarizes the autoregressive flow matching training procedure.

Algorithm 1 Training: Autoregressive Flow Matching.

Require: Dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$, window size w , networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
- 2: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$ and $(\mathbf{y}_t^1, \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \sim \mathcal{D}$
- 3: Set $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$
- 4: Sample $s \sim \mathcal{U}(0, 1)$ and $\mathbf{y}_t^s \sim \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I)$
- 5: Compute target: $\mu(\mathbf{y}_t^s, s|\mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$
- 6: Compute loss: $\mathcal{L}_t = \|\mu(\mathbf{y}_t^s, s|\mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2$
- 7: Update θ, ϕ
- 8: **end while**
- 9: **return** ν_θ, ζ_ϕ

Algorithm 2 Sampling: Autoregressive Flow Matching.

Require: Past observations $\mathbf{Y}_l$, covariates $\mathbf{C}$, forecast horizon f , window size w , networks ν_θ, ζ_ϕ

- 1: Initialize $\mathbf{Y}_f \leftarrow \mathbf{Y}_l$
- 2: **for** $t = \tau + 1$ **to** $\tau + f$ **do**
- 3: Set context: $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$, using the last w entries in $\mathbf{Y}_f$
- 4: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$
- 5: Solve $\frac{d\psi(\mathbf{y}_t, s)}{ds} = \nu_\theta(\psi(\mathbf{y}_t, s), \mathbf{h}_t, \mathbf{c}_t, s), \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t^0, \quad s \in [0, 1]$
- 6: Append $\psi(\mathbf{y}_t, 1)$ to $\mathbf{Y}_f$
- 7: **end for**
- 8: **return** $\mathbf{Y}_f$

3.3 INFERENCE

After training, we wish to provide probabilistic predictions for new observations $\mathbf{Y}_l$ and corresponding covariates $\mathbf{C}$ for f future timestamps into the future. For this we follow the sampling procedure in Algorithm 2, where predictions are made autoregressively using a rolling-window approach. This procedure can be repeated many times to obtain empirical quantiles of the uncertainty of our predictions.

4 EXPERIMENTS

In this section, we present our empirical results and compare FLOWTIME against various baselines using real world datasets and data generated via simulating various dynamical systems.

4.1 DYNAMICAL SYSTEMS

Data We evaluated FLOWTIME on the task of forecasting five different stochastic dynamical systems: stochastic versions of the Lorenz system (Lorenz, 1963), FitzHugh–Nagumo model (FitzHugh, 1961), Lotka-Volterra system Lotka (1925), Brusselator (Lefever & Prigogine, 1968), and Van der Pol oscillator (van der Pol, 1926). For each system, we generated trajectories by numerically solving the corresponding stochastic differential equations, using randomly sampled initial conditions and Brownian motion realizations. The details of the dynamical systems and the data generation process are provided in Appendix A.1.

Implementation details For these experiments we set ζ_ϕ as a two-layer bi-directional LSTM with 64 hidden units, and set ν_θ as an MLP with three hidden layers of size 64. We use Fourier positional embeddings to encode the flow step s into a 16-dimensional vector using a fixed set of frequencies and set the context length w to be equal to the prediction length. We trained our models via stochastic gradient descent using an Adam optimizer with a learning rate of 0.003 and a batch size of 128. Training until convergence took ~ 7 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines To evaluate the effectiveness of the autoregressive factorization in FLOWTIME, we compared our approach against a non-autoregressive flow-matching baseline, which directly models the joint distribution of future observations conditioned on past observations and covariates, i.e.

$p^1 = p(\mathbf{Y}_f | \mathbf{Y}_l, \mathbf{C})$. This approach serves as a direct ablation, isolating the effect of autoregressive factorization introduced in FLOWTIME. Note that this non-autoregressive formulation closely resembles the method proposed in (Kollovieh et al., 2024). Further details on this baseline are provided in Appendix B.

Evaluation To assess the model’s ability to extrapolate beyond its training distribution, we partitioned each trajectory into three segments: (1) an observation window, which serves as the historical input to the model; (2) a prediction window, which is available only during training to act as the target; and (3) an extrapolation window, which remains entirely unseen during training. We used the continuous rank probability score (CRPS) (Winkler et al., 1996) for evaluating the performance of the model in uncertainty quantification and the normalized root mean square error (NRMSE) (Hyndman & Koehler, 2006) to assess the accuracy of the predictions.

Table 1: Performance comparison of autoregressive (FLOWTIME) vs non-autoregressive (Non-AR) factorization in our method for five different stochastic dynamical systems. Performance is compared in terms of CRPS and NRMSE for both prediction and extrapolation regimens on the test set. Reported results are the mean and standard deviation across five different random seeds.

System	Factorization	Prediction		Extrapolation	
		CRPS ↓	NRMSE ↓	CRPS ↓	NRMSE ↓
Lorenz	Non-AR	0.147±0.012	0.242±0.018	0.120 ±0.009	0.301±0.024
	FLOWTIME	0.090 ±0.008	0.017 ±0.003	0.137±0.011	0.048 ±0.006
FitzHugh-Nagumo	Non-AR	0.165±0.014	0.131±0.011	0.214±0.016	0.306±0.022
	FLOWTIME	0.144 ±0.013	0.090 ±0.007	0.146 ±0.012	0.278 ±0.019
Lotka-Volterra	Non-AR	0.130±0.011	0.266±0.019	0.136±0.012	0.502±0.033
	FLOWTIME	0.126 ±0.010	0.189 ±0.015	0.123 ±0.009	0.339 ±0.026
Brusselator	Non-AR	0.217±0.016	0.658±0.042	0.238±0.019	1.046±0.057
	FLOWTIME	0.125 ±0.011	0.031 ±0.004	0.109 ±0.008	0.023 ±0.003
Van der Pol	Non-AR	0.143 ±0.012	0.161±0.014	0.211 ±0.018	0.510±0.031
	FLOWTIME	0.225±0.017	0.051 ±0.005	0.282±0.021	0.101 ±0.009

Results Our results, summarized in Table 1, demonstrates that the autoregressive factorization employed in FLOWTIME significantly improves forecasting performance across multiple stochastic dynamical systems. For the prediction task, the AR approach outperforms the non-AR baseline in terms of NRMSE across all five systems, with particularly dramatic improvements observed in the Lorenz (0.017 vs 0.242) and Brusselator (0.031 vs 0.658) systems, representing reductions in error of 93% and 95%, respectively. The AR factorization also yields superior CRPS in four of the five systems, with only the Van der Pol oscillator showing better uncertainty quantification with the non-AR approach. More importantly, these improvements extend to the extrapolation regime, where autoregressive factorization maintains its advantage in NRMSE across all systems, demonstrating its superior ability to generalize beyond the training distribution. This is particularly evident in the Brusselator system, where AR factorization reduces the extrapolation NRMSE by 98% (0.023 vs 1.046). These results confirm that modeling the temporal dependencies through autoregressive factorization enables FLOWTIME to better capture the underlying dynamics of complex stochastic systems, leading to more accurate predictions and improved uncertainty quantification, especially in challenging extrapolation scenarios.

4.2 REAL-WORLD DATASETS

Data We evaluated FLOWTIME on the tasks of forecasting multiple univariate and multivariate timeseries with different frequencies (hourly and daily) from GluonTS (Alexandrov et al., 2020). Specifically, we used the following datasets: Electricity (Lai et al., 2018), Solar (Lai et al., 2018), Exchange (Dua et al., 2017), Traffic (Dua et al., 2017), and Wikipedia (Gasthaus et al., 2019). We provide further details in Appendix A.2.

Implementation details We set ζ_ϕ as a three-layer bi-directional LSTM with 64 hidden units. We defined ν_θ via a residual neural network with five residual blocks implemented via 1-D convolutional layers with gated activation functions (Van Den Oord et al., 2016; Kong et al., 2020). The architecture is similar to the network used in (Rasul et al., 2021) but with fewer residual blocks. We used Fourier positional embeddings to encode the flow step s into a 32-dimensional vector using a fixed set of frequencies. We used the same embedding technique to encode time-dependent covariates c . We trained our models via stochastic gradient descent using an Adam optimizer with a learning rate of 0.001 and a batch size of 128. Depending on the dataset, training until convergence took from 40 – 100 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Baselines We compared FLOWTIME against multiple established baselines. These include traditional statistical methods such as Seasonal Naive (SN), AutoARIMA, and AutoETS (Hyndman et al., 2008), as well as deep learning methods such as DeepAR (Salinas et al., 2020), WaveNet (Van Den Oord et al., 2016). We further include diffusion baselines such as TSDiff (Kolloviev et al., 2023), SSSD (Alcaraz & Strodthoff, 2022), and TimeGrad (Rasul et al., 2021) and a flow matching baseline TSFlow (Kolloviev et al., 2024).

Table 2: Test set CRPS comparison (lower is better) of different models on forecasting of five real world data sets. Mean and standard error metrics for FlowTime obtained by re-training and evaluating five times. Results for the baselines are from (Kolloviev et al., 2024). Best scores in **bold**, second best underlined.

Method	Electr.	Exchange	Solar	Traffic	Wikipedia
SN	0.060±0.000	0.013±0.000	0.512±0.000	0.221±0.000	0.423±0.000
ARIMA	0.344±0.000	0.013±0.000	0.558±0.003	0.486±0.000	0.654±0.000
ETS	0.056±0.000	0.008±0.000	0.550±0.000	0.492±0.000	0.651±0.000
DeepAR	0.051±0.000	0.013±0.004	0.429±0.015	0.103±0.000	0.215±0.003
WaveNet	0.058±0.008	0.012±0.001	0.360±0.000	0.099±0.000	0.207±0.000
CSDI	0.051±0.000	0.013±0.000	0.360±0.000	0.152±0.000	0.318±0.012
SSSD	0.048±0.000	0.010±0.000	0.354±0.024	0.107±0.002	0.209±0.000
TSFlow	0.045±0.001	0.009±0.001	0.343±0.002	0.083±0.000	0.227±0.000
FLOWTIME	0.042±0.001	0.009±0.001	0.284±0.002	<u>0.089±0.000</u>	0.243±0.002

Results Table 2 presents the comparison of our proposed FLOWTIME model against various baselines on five real-world timeseries datasets. The results demonstrate that FLOWTIME achieves state-of-the-art performance on three of the five datasets: Electricity, Exchange, and Solar. On the Electricity dataset, FLOWTIME outperforms all baselines with a CRPS of 0.042, representing a 6.7% improvement over the second-best model, TSFlow (0.045). For the Exchange dataset, FLOWTIME matches the performance of TSFlow with a CRPS of 0.009, significantly outperforming traditional statistical methods and other deep learning approaches. Most notably, on the Solar dataset, FLOWTIME achieves a substantial improvement with a CRPS of 0.284, representing a 17.2% reduction in error compared to the next best model, TSFlow (0.343). While FLOWTIME ranks second on the Traffic dataset with a CRPS of 0.089, slightly behind TSFlow (0.083), it still outperforms all other traditional and deep learning baselines. On the Wikipedia dataset, FLOWTIME achieves a competitive CRPS of 0.243, positioning it behind only WaveNet (0.207) and SSSD (0.209). These results demonstrate that the autoregressive flow-matching approach employed in FLOWTIME consistently delivers superior or competitive performance across diverse real-world timeseries forecasting tasks.

5 DISCUSSION

In this work, we introduced FLOWTIME, a probabilistic forecasting model that combines autoregressive modeling with flow matching to generate accurate and well-calibrated predictions for multivariate timeseries data. By factorizing the forecasting problem into a sequence of conditional distributions, FLOWTIME effectively balances the sequential dependencies in temporal data with the expressiveness of flow-based generative modeling. Our empirical results across simulated dynamical systems and real-world datasets demonstrate that FlowTime achieves competitive performance with state-of-the-art baselines in challenging forecasting benchmarks. The autoregressive decomposition leads to notable improvements in uncertainty calibration and distribution coverage compared to

standard flow-matching approaches that model entire future windows simultaneously. Furthermore, FLOWTIME exhibits robust generalization to out-of-distribution scenarios, a critical capability for real-world deployment where test conditions often diverge from training data.

With that said, FLOWTIME has limitations that warrant discussion. First, the sequential nature of autoregressive sampling introduces computational overhead during inference compared to fully parallelizable non-autoregressive models, which may limit its applicability in real-time forecasting systems. Additionally, our current implementation applies flow matching directly in the data space, which may be suboptimal for high-dimensional partially observed data. Future work could explore latent-space formulations, extend to irregularly sampled data, and investigate broader applications in scientific modeling and decision-making under uncertainty. Overall, FLOWTIME represents a step forward in probabilistic timeseries forecasting, demonstrating the potential of flow-based generative models for improving both accuracy and uncertainty quantification.

REFERENCES

- Michael S Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv preprint arXiv:2303.08797*, 2023.
- Juan Miguel Lopez Alcaraz and Nils Strodthoff. Diffusion-based time series imputation and forecasting with structured state space models. *arXiv preprint arXiv:2208.09399*, 2022.
- Alexander Alexandrov, Konstantinos Benidis, Michael Bohlke-Schneider, Valentin Flunkert, Jan Gasthaus, Tim Januschowski, Danielle C Maddix, Syama Rangapuram, David Salinas, Jasper Schulz, et al. GluonTS: Probabilistic and neural time series modeling in python. *Journal of Machine Learning Research*, 21(116):1–6, 2020.
- Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.
- Fatoumata Dama and Christine Sinoquet. Time series analysis and modeling to forecast: a survey. In *ArXiv:2104.00164*, 2021.
- Abhyuday Desai, Cynthia Freeman, Zuhui Wang, and Ian Beaver. TimeVAE: A variational auto-encoder for multivariate time series generation. *arXiv preprint arXiv:2111.08095*, 2021.
- Dheeru Dua, Casey Graff, et al. UCI machine learning repository. URL <http://archive.ics.uci.edu/ml>, 7(1):62, 2017.
- Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first International Conference on Machine Learning*, 2024.
- Richard FitzHugh. Impulses and physiological states in theoretical models of nerve membrane. *Biophysical Journal*, 1(6):445–466, 1961. doi: 10.1016/S0006-3495(61)86902-6.
- Jan Gasthaus, Konstantinos Benidis, Yuyang Wang, Syama Sundar Rangapuram, David Salinas, Valentin Flunkert, and Tim Januschowski. Probabilistic forecasting with spline quantile function RNNs. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pp. 1901–1910. PMLR, 2019.
- Will Grathwohl, Ricky TQ Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. FFJORD: Free-Form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations*, 2018.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.
- Yang Hu, Xiao Wang, Lirong Wu, Huatian Zhang, Stan Z. Li, Sheng Wang, and Tianlong Chen. Fm-ts: Flow matching for time series generation. In *ArXiv:2411.07506v1 [cs.LG]*, 2024. URL <http://arxiv.org/abs/2411.07506>.

432 Rob Hyndman, Anne B Koehler, J Keith Ord, and Ralph D Snyder. *Forecasting with Exponential*
433 *Smoothing: The State Space Approach*. Springer Science & Business Media, 2008.

434 Rob J Hyndman and Anne B Koehler. Another look at measures of forecast accuracy. *International*
435 *Journal of Forecasting*, 22(4):679–688, 2006.

436 Maximilian Karl, Maximilian Soelch, Justin Bayer, and Patrick Van der Smagt. Deep varia-
437 tional Bayes filters: Unsupervised learning of state space models from raw data. *arXiv preprint*
438 *arXiv:1605.06432*, 2016.

439 Marcel Kollovich, Abdul Fatir Ansari, Michael Bohlke-Schneider, Jasper Zschiegner, Hao Wang,
440 and Yuyang Bernie Wang. Predict, refine, synthesize: Self-guiding diffusion models for proba-
441 bilistic time series forecasting. *Advances in Neural Information Processing Systems*, 36:28341–
442 28364, 2023.

443 Marcel Kollovich, Marten Lienen, David Lüdke, Leo Schwinn, and Stephan Günnemann. Flow
444 matching with gaussian process priors for probabilistic time series forecasting. In *The Thirteenth*
445 *International Conference on Learning Representations*, 2024.

446 Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. DiffWave: A versatile
447 diffusion model for audio synthesis. *arXiv preprint arXiv:2009.09761*, 2020.

448 Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long- and short-term
449 temporal patterns with deep neural networks. In *The 41st International ACM SIGIR Conference*
450 *on Research & Development in Information Retrieval*, pp. 95–104, 2018.

451 R. Lefever and I. Prigogine. Symmetry breaking instabilities in dissipative systems. II. *The Journal*
452 *of Chemical Physics*, 48(4):1695–1700, 1968. doi: 10.1063/1.1668896.

453 Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: A survey. *Philosophical*
454 *Transactions of the Royal Society A*, 379(2194):20200209, 2021.

455 Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow match-
456 ing for generative modeling. In *The Eleventh International Conference on Learning Representa-*
457 *tions*, 2022.

458 Xingchao Liu, Chengyue Gong, et al. Flow straight and fast: Learning to generate and transfer data
459 with rectified flow. In *The Eleventh International Conference on Learning Representations*, 2022.

460 Edward Lorenz. Deterministic nonperiodic flow. *Journal of Atmospheric Sciences*, 20(2), 1963.

461 Alfred J. Lotka. *Elements of Physical Biology*. Williams & Wilkins, 1925.

462 Caspar Meijer and Lydia Y Chen. The rise of diffusion models in time-series forecasting. *arXiv*
463 *preprint arXiv:2401.03006*, 2024.

464 Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, Ann Lee, Apoorv
465 Vyas, Bowen Shi, Chih-Yao Ma, Ching-Yao Chuang, et al. MovieGen: A cast of media founda-
466 tion models. *arXiv e-prints*, pp. arXiv–2410, 2024.

467 Kashif Rasul, Abdul-Saboor Sheikh, Ingmar Schuster, Urs Bergmann, and Roland Vollgraf. Mul-
468 tivariate probabilistic time series forecasting via conditioned normalizing flows. *arXiv preprint*
469 *arXiv:2002.06103*, 2020.

470 Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. Autoregressive denoising dif-
471 fusion models for multivariate probabilistic time series forecasting. In *International Conference*
472 *on Machine Learning*, pp. 8857–8868. PMLR, 2021.

473 David Salinas, Michael Bohlke-Schneider, Laurent Callot, Roberto Medico, and Jan Gasthaus.
474 High-dimensional multivariate forecasting with low-rank gaussian copula processes. *Advances*
475 *in Neural Information Processing Systems*, 32, 2019.

476 David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic
477 forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3):
478 1181–1191, 2020.

486 Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised
487 learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning*,
488 pp. 2256–2265. PMLR, 2015.

489 Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben
490 Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint*
491 *arXiv:2011.13456*, 2020.

492
493 Ella Tamir, Najwa Laabid, Markus Heinonen, Vikas Garg, and Arno Solin. Conditional flow match-
494 ing for time series modelling. In *ICML Workshop: Structured Probabilistic Inference and Gener-
495 ative Modeling*, 2024.

496 Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. CSDI: Conditional score-based
497 diffusion models for probabilistic time series imputation. *Advances in Neural Information Pro-
498 cessing Systems*, 34:24804–24816, 2021.

499 Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Huguet, Yanlei Zhang, Jarrod Rector-
500 Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models
501 with minibatch optimal transport. *arXiv preprint arXiv:2302.00482*, 2023.

502
503 Aaron Van Den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves,
504 Nal Kalchbrenner, Andrew Senior, Koray Kavukcuoglu, et al. WaveNet: A generative model for
505 raw audio. *arXiv preprint arXiv:1609.03499*, 12, 2016.

506
507 B. van der Pol. On ‘relaxation-oscillations’. *The London, Edinburgh, and Dublin Philosophical*
508 *Magazine and Journal of Science*, 2(11):978–992, 1926. doi: 10.1080/14786442608564127.

509 Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent
510 neural networks. *Neural Computation*, 1(2):270–280, 1989.

511
512 Robert L Winkler, Javier Munoz, José L Cervera, José M Bernardo, Gail Blattenberger, Joseph B
513 Kadane, Dennis V Lindley, Allan H Murphy, Robert M Oliver, and David Ríos-Insua. Scoring
514 rules and the evaluation of probabilities. *Test*, 5:1–60, 1996.

515 Jiexia Ye, Weiqi Zhang, Ke Yi, Yongzi Yu, Ziyue Li, Jia Li, and Fugee Tsung. A survey of time
516 series foundation models: Generalizing time series representation with large language model. In
517 *ArXiv:2104.00164*, 2024.

518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539

A DATASETS

A.1 DYNAMICAL SYSTEMS

We generated training and testing data for five classical stochastic dynamical systems, that is, Lorenz, FitzHugh-Nagumo, Lotka-Volterra, Brusselator and Van der Pol, by numerically integrating their corresponding stochastic differential equations (SDEs). Each system has the general form

$$d\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t; \Theta) dt + \sigma d\mathbf{W}(t),$$

where, f is the deterministic drift, Θ are the system parameters, σ is a constant diffusion term, and $\mathbf{W}(t)$ is a standard Brownian motion. We sampled 2400 random initial conditions from a uniform distribution over the specified range (see Table 3) for each system and generated a trajectory for each initial condition by solving the SDE using the Euler-Heun method with fixed step size. We sampled 200 equally spaced time points for each trajectory. Out of the 200 time points in each trajectory, the first 75 were used as observed data, the next 75 for prediction, and the final 50 for extrapolation. Out of the 2400 trajectories generated, we used 2000 for training and 400 for testing.

Table 3: Summary of stochastic dynamical systems. $\mathbf{f}(\mathbf{x})$ denotes the drift function, Θ the system parameters, σ the constant diffusion term, and $p(\mathbf{x}(0))$ the initial condition distribution. Each system was integrated over the given interval $[t_0, t_1]$ with 200 steps using the Euler-Heun method.

System	$f(\mathbf{x})$	Θ	Σ	$p(\mathbf{x}(0))$	$[t_0, t_1]$
Lorenz	$f_1 = \sigma(x_2 - x_1)$	$\sigma = 10$	(1.5, 1.5, 1.5)	$\mathcal{U}([0, 10])$	[0, 2]
	$f_2 = x_1(\rho - x_3) - x_2$	$\rho = 28$			
	$f_3 = x_1 x_2 - \beta x_3$	$\beta = \frac{8}{3}$			
FitzHugh-Nagumo	$f_1 = x_1 - \frac{x_1^3}{3} - x_2 + I$	$a = 0.7$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 10]
	$f_2 = \frac{x_1 + a - b x_2}{\tau}$	$b = 0.8$			
		$\tau = 12.5$			
Lotka-Volterra	$f_1 = \alpha x_1 - \beta x_1 x_2$	$\alpha = 1.3$	(1.5, 1.5)	$\mathcal{U}([0, 5])$	[0, 20]
	$f_2 = -\delta x_2 + \gamma x_1 x_2$	$\beta = 0.9$			
		$\gamma = 0.8$			
Brusselator	$f_1 = A + x_1^2 x_2 - (B + 1) x_1$	$A = 1.0$	(1.5, 1.5)	$\mathcal{U}([0, 2])$	[0, 20]
	$f_2 = B x_1 - x_1^2 x_2$	$B = 3.0$			
		$\delta = 1.8$			
Van der Pol	$f_1 = x_2$	$\mu = 0.1$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 20]
	$f_2 = \mu(1 - x_1^2) x_2 - x_1$				

A.2 REAL-WORLD DATASETS

We used five different datasets from GluonTS (Alexandrov et al., 2020) to evaluate our model performance. Specifically, we used the versions preprocessed as in (Salinas et al., 2019). A summary of their properties is listed in Table 4.

Table 4: Properties of the datasets used in the experiments: dimension n , domain Ω , frequency, total training timesteps, and prediction length.

Dataset	n	Ω	freq.	timesteps	prediction length
Electricity	370	$\mathbb{R}^+$	hourly	5833	24
Exchange	8	$\mathbb{R}^+$	daily	6071	30
Solar	137	$\mathbb{R}^+$	hourly	7009	24
Traffic	963	(0, 1)	hourly	4001	24
Wikipedia	2000	$\mathbb{N}$	daily	792	30

B NON-AUTOREGRESSIVE FLOW MATCHING BASELINE

Problem Setting We consider the same forecasting setup as Section 3 where given past observations $\mathbf{Y}_l$ and covariates $\mathbf{C}$, our goal remains to model the conditional distribution $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$. However, unlike the autoregressive factorization, we directly model the joint distribution of the entire future trajectory $\mathbf{Y}_f \in \mathbb{R}^{f \times n}$ without temporal factorization:

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = p(\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}|\mathbf{Y}_l, \mathbf{C}) \quad (9)$$

This formulation preserves temporal correlations across all future time steps but requires learning a high-dimensional distribution.

Training We construct a probability path $(p^s(\mathbf{Y}_f))^{0 \leq s \leq 1}$ that transports samples from a prior distribution $p^0(\mathbf{Y}_f) = \mathcal{N}(0, \Sigma)$ to the target distribution $p^1(\mathbf{Y}_f) \approx p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$, where Σ is a block-diagonal covariance matrix. The prior can be interpreted as independent Brownian motion processes per dimension, where Brownian motion is defined as a stochastic process $W: [\tau+1, \tau+f] \rightarrow \mathbb{R}^n$.

We define the conditional probability path using a linear interpolation bridge with Brownian noise:

$$p^s(\mathbf{Y}_f|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma) \quad (10)$$

where $\mathbf{z} = (\mathbf{Y}_f^0, \mathbf{Y}_f^1)$ with $\mathbf{Y}_f^0 \sim p^0$ and $\mathbf{Y}_f^1 \sim p^1$. The corresponding conditional velocity field becomes:

$$\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0 + \frac{\sigma^2(1-2s)}{2}\Sigma^{-1}(\mathbf{Y}_f - ((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1)) \quad (11)$$

When $\sigma^2 \rightarrow 0$, this reduces to the straight path velocity $\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0$. We learn a neural velocity field ν_θ that operates on the entire future trajectory. The context encoding remains similar to Eq. (7):

$$\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f}) \quad (12)$$

where ζ_ϕ now processes both past and future covariates. The training objective becomes:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{Y}_f^s \sim p^s(\mathbf{Y}_f|\mathbf{z})} \|\mu(\mathbf{Y}_f^s, s|\mathbf{z}) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|^2 \quad (13)$$

where ν_θ is implemented as a sequential neural network (e.g., Transformer or RNN) that processes the entire trajectory. Sampling requires solving the trajectory-level ODE:

$$\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s), \quad \psi(\mathbf{Y}_f, 0) \sim p^0 \quad (14)$$

using numerical solvers. This produces joint samples from $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ without autoregressive decomposition.

Implementation Details We used this baseline to conduct experiments on the task of forecasting multiple dynamical systems. For this setup we set ν_θ as a 4-layer bi-directional LSTM with 128 hidden units. We kept the same training parameters as in Section 4.1. Training until convergence took ~ 2 minutes on a single A100-SXM4-40GB NVIDIA GPU.

Key Differences from Autoregressive Variant The non-autoregressive approach differs fundamentally from its autoregressive counterpart introduced in FLOWTIME. First, while the autoregressive method factorizes the joint distribution via a Markovian decomposition across time steps, the non-autoregressive baseline directly models the full future trajectory $\mathbf{Y}_f$ as a single high-dimensional random variable. This preserves cross-temporal dependencies at the expense of learning a more complex distribution over $\mathbb{R}^{n \times f}$. Second, computational characteristics diverge significantly: the non-autoregressive version processes entire trajectories through sequential neural networks for the velocity fields enabling parallel generation of all future timepoints at the cost of higher memory requirements. Finally, their uncertainty propagation mechanisms contrast sharply: the autoregressive approach explicitly models how prediction errors accumulate through the chain of conditional distributions, while the non-autoregressive method captures joint uncertainty over all timesteps through trajectory-level sampling but lacks explicit mechanisms to model error accumulation dynamics. These differences create complementary trade-offs – the autoregressive variant offers interpretable uncertainty quantification and efficient window-based computation but assumes limited Markovian dependencies, while the non-autoregressive baseline preserves full temporal correlations at higher computational cost with less explicit uncertainty dynamics.

Algorithm 3 Non-Autoregressive Flow Matching: Training

Require: Dataset $\mathcal{D}$, networks ν_θ, ζ_ϕ
1: **while** not converged **do**
2: Sample $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$ and $(\mathbf{Y}_f^1, \mathbf{Y}_l, \mathbf{C}) \sim \mathcal{D}$
3: Set $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f})$
4: Sample $s \sim \mathcal{U}(0, 1)$
5: $\mathbf{Y}_f^s \sim \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma)$
6: Compute target $\mu(\mathbf{Y}_f^s, s | \mathbf{z})$
7: Update θ, ϕ to minimize $\|\mu(\cdot) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|_{\Sigma^{-1}}^2$
8: **end while**

Algorithm 4 Non-Autoregressive Flow Matching: Sampling

Require: $\mathbf{Y}_l, \mathbf{C}$, networks ν_θ, ζ_ϕ
1: Encode context: $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C})$
2: Sample initial trajectory $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$
3: Solve $\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)$
4: Return $\psi(\mathbf{Y}_f, 1)$

C EVALUATION METRICS

In this section, we describe the metrics used to evaluate the quality of our forecasts. Given an n -dimensional time series $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$ and its predictive distribution or samples from our probabilistic model, we employ two metrics: the Continuous Ranked Probability Score (CRPS) and the Normalized Root Mean Squared Error (NRMSE).

C.1 CONTINUOUS RANKED PROBABILITY SCORE (CRPS)

The CRPS is a proper scoring rule commonly used to assess the quality of probabilistic forecasts. For a univariate random variable X with cumulative distribution function (CDF) F_X and an observed value x , the CRPS is defined as:

$$\text{CRPS}(F_X, x) = \int_{-\infty}^{+\infty} (F_X(y) - \mathbf{1}\{y \geq x\})^2 dy, \quad (15)$$

where $\mathbf{1}\{\cdot\}$ is the indicator function. Intuitively, the CRPS evaluates how well the entire predictive distribution aligns with the observation x . Lower CRPS values indicate better-calibrated and more accurate predictive distributions.

In the context of time series forecasting, we can compute the CRPS for each time step in the forecasting horizon and average the results:

$$\text{CRPS}_{\text{avg}} = \frac{1}{f} \sum_{k=1}^f \text{CRPS}(F_{X_{\tau+k}}, y_{\tau+k}), \quad (16)$$

where $F_{X_{\tau+k}}$ is the predicted CDF (or an empirical CDF from samples) at forecast horizon k , and $y_{\tau+k}$ is the corresponding ground-truth observation.

C.2 NORMALIZED ROOT MEAN SQUARED ERROR (NRMSE)

The Root Mean Squared Error (RMSE) is a standard metric for evaluating the accuracy of point forecasts. For a set of predictions $\{\hat{y}_{\tau+1}, \dots, \hat{y}_{\tau+f}\}$ and corresponding ground-truth values $\{y_{\tau+1}, \dots, y_{\tau+f}\}$, the RMSE is:

$$\text{RMSE} = \sqrt{\frac{1}{f} \sum_{k=1}^f (\hat{y}_{\tau+k} - y_{\tau+k})^2}. \quad (17)$$

To make the RMSE scale-invariant and facilitate comparison across different datasets or time series with different magnitudes, we use the Normalized RMSE (NRMSE). One common normalization is

by the standard deviation of the ground-truth series, σ_y :

$$\text{NRMSE} = \frac{\text{RMSE}}{\sigma_y}, \quad (18)$$

where σ_y is the sample standard deviation of the observations. Alternatively, one could normalize by the range of the data ($\max(y) - \min(y)$) depending on the application. A lower NRMSE indicates more accurate predictions relative to the variability of the time series.